

Highlights

A Reliability-Oriented Pipeline for Natural-Language-to-Service-Level IoT Automation

Mingi Jeong, EunJin Jeong, Soonhoi Ha

- Service-aware retrieval narrows the action space for IoT program generation.
- Deterministic checks block invalid services, types, and temporal behavior.
- The proposed pipeline reaches a 0.98 pass rate on JOICommands-170.
- Prompt tokens fall by 96% and end-to-end latency by 35%.

A Reliability-Oriented Pipeline for Natural-Language-to-Service-Level IoT Automation

Mingi Jeong^{a,b}, EunJin Jeong^b and Soonhoi Ha^{b,*}

^aElectronics and Telecommunications Research Institute, 218 Gajeong-ro, Yuseong-gu, Daejeon, 34129, Republic of Korea

^bDepartment of Computer Science and Engineering, Seoul National University, 1 Gwanak-ro, Gwanak-gu, Seoul, 08826, Republic of Korea

ARTICLE INFO

Keywords:

JOILang
IoT automation
domain-specific languages
large language models
program generation
deterministic validation

ABSTRACT

Natural-language control can lower the barrier to Internet of Things (IoT) automation, but directly generating executable rules with large language models (LLMs) can produce hallucinated services, invalid APIs, unsafe actions, and incorrect temporal behavior. This paper studies reliable natural-language-to-IoT program generation using JOILang as a service-level executable representation. JOILang provides tag-based scoping, runtime late binding, and explicit temporal and policy constraints, while the proposed pipeline addresses the translation problem above this representation layer. The pipeline combines service pre-mapping, retrieval-augmented prompt construction, state-aware skeleton generation for temporally complex commands, deterministic validation and repair, and reverse translation for user-side inspection. We evaluate five prompting strategies on JOICommands-170, a benchmark of 170 smart-home and smart-office commands paired with expert-curated JOILang reference programs. The evaluation combines deterministic admissibility checking with semantic assessment against the reference set. The best configuration, *Pre-map-CoT+RAG*, improves pass rate from 0.62 to 0.98 and deterministic quality from 0.71 to 0.98 compared with a full-prompt baseline, while reducing average input tokens by approximately 96% and end-to-end latency by approximately 35%. These results indicate that service-aware context narrowing, structured prompting, and deterministic validation jointly improve the reliability and efficiency of LLM-assisted IoT automation within the evaluated smart-environment setting.

1. Introduction

Internet of Things (IoT) automation increasingly involves heterogeneous sensors, actuators, cloud services, and user-defined policies in homes, offices, and other user-centric environments [1]. Existing trigger-action programming tools and macroprogramming frameworks lower the entry barrier, but users still need to reason about device capabilities, platform-specific APIs, and the precise conditions under which a rule should execute [2–6]. Interoperability standards and capability models, including the W3C Web of Things, Matter, EUPont, and SmartThings, improve syntactic and semantic descriptions of devices [7–10]. They do not, however, by themselves provide a reliable natural-language path from an end-user instruction to an executable automation program.

Large language models (LLMs) make this path more plausible because they can translate informal requirements into code-like outputs [11, 12]. For IoT automation, however, direct generation is risky. A generated program may call a service that does not exist in the current environment, bind an action to the wrong device scope, confuse a persistent state with a transition event, or produce a syntactically plausible rule that violates execution or policy constraints. These failures are more consequential than ordinary text-generation errors because the output may be executed by physical devices.

This paper studies this reliability problem in user-centric smart homes and smart offices. The study does not target mission-critical industrial IoT or hard real-time control, where certification, fault tolerance, and timing guarantees require a different evidence standard. Instead, it focuses on soft real-time automation scenarios in which non-expert users express intended behavior in natural language and the system must generate an executable, inspectable, and policy-compatible IoT program.

We use JOILang as the target representation. JOILang is a service-level IoT domain-specific language (DSL) grounded in prior service-oriented IoT work [13, 14]. Instead of binding rules to concrete device identifiers, JOILang

*Corresponding author.

 mgjeong@etri.re.kr (M. Jeong); chjej202@snu.ac.kr (E. Jeong); sha@snu.ac.kr (S. Ha)

ORCID(s): 0000-0001-9287-9726 (M. Jeong); 0000-0002-9585-3369 (E. Jeong); 0000-0001-7472-9142 (S. Ha)

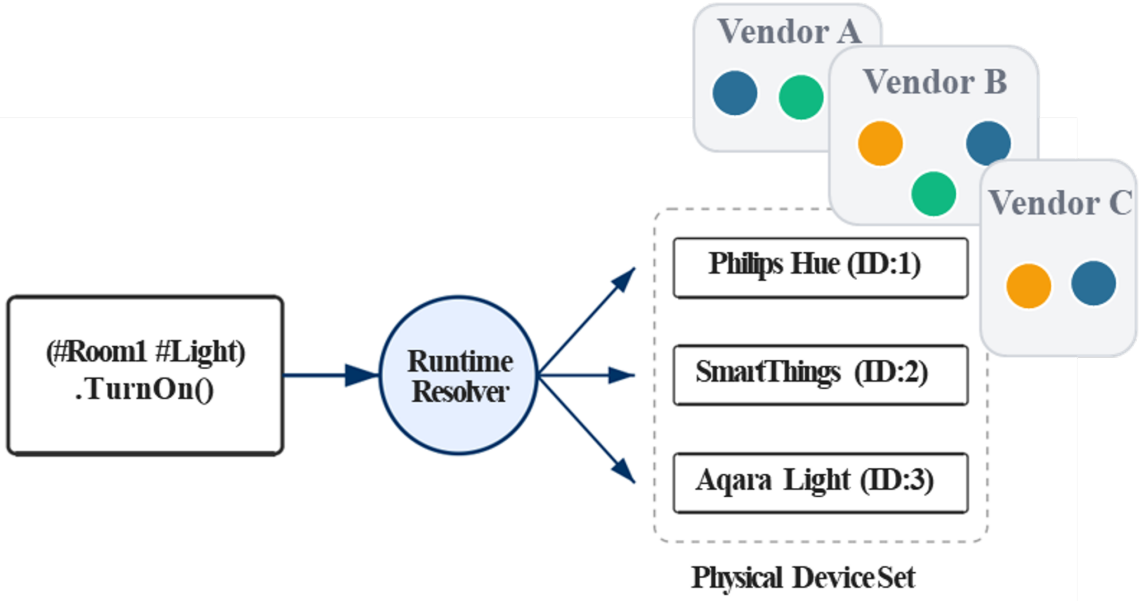


Figure 1: Conceptual illustration of JOILang’s service-level late binding. A logical tag-based command is resolved at runtime to the currently available physical device set.

describes which service should be applied to a tag-defined scope. For example, `(#Room1 #Light).TurnOn()` denotes a logical command over a room and device type rather than a vendor-specific device ID. At runtime, a resolver maps the tag-level command to the currently available physical devices (Fig. 1), allowing the same scenario to remain stable when devices are added, removed, or replaced.

The main contribution of this paper is not the JOILang representation alone, but the reliability-oriented translation pipeline built above it. The pipeline combines service pre-mapping for schema narrowing, retrieval-augmented prompt construction, state-aware skeleton generation for temporally complex commands, deterministic validation and repair, and reverse translation for user-side inspection. This design treats LLM generation as one component of a constrained automation process rather than as an unconstrained end-to-end code generator.

We evaluate the approach on *JOICommands-170*, a dataset of 170 English commands paired with expert-curated JOILang reference programs for smart-home and smart-office scenarios. Candidates are judged by deterministic admissibility checks and by semantic comparison against the expert-curated references. Under this protocol, the best configuration, Pre-map-CoT+RAG (service pre-mapping with chain-of-thought prompting and retrieval-augmented generation), reaches a pass rate of 0.98 and a deterministic quality score of $S_{\text{DET}} = 0.98$.

The contributions of this study are summarized as follows:

- We formulate reliable natural-language-to-IoT program generation using JOILang as a service-level executable representation with tag-based scoping, runtime late binding, temporal constructs, and policy-aware execution semantics.
- We design a reliability-oriented NL-to-JOILang pipeline that integrates service pre-mapping, retrieval-augmented prompt construction, state-aware skeleton generation, deterministic validation and repair, and reverse translation for user-side inspection.
- We introduce JOICommands-170 and a hybrid evaluation protocol that combines deterministic validation with semantic assessment against expert-curated reference programs.
- We empirically compare five prompting strategies and component-removed variants, showing that service-aware context narrowing and structured prompting improve both reliability and efficiency in the evaluated smart-environment setting.

The rest of this paper is organized as follows. Section 2 reviews related work on natural-language IoT automation and constraint-guided code generation. Section 3 introduces JOILang and its execution semantics. Section 4 presents the proposed reliability-oriented NL-to-JOILang pipeline. Section 5 reports the experimental results on JOICommands-170. Section 6 discusses the findings and limitations, and Section 7 concludes the paper.

2. Related Work

Natural-language-driven IoT automation lies at the intersection of end-user IoT programming platforms, natural-language-to-code (NL-to-code) and NL-to-DSL generation, constraint-guided LLM decoding, and edge-cloud IoT runtimes. This section positions the proposed pipeline against representative research lines and identifies the gap that motivates a validation-guided NL-to-IoT translation process.

2.1. Natural-Language Interfaces to IoT Automation and Scripts

Early smart-home automation systems popularized trigger-action programming (TAP), where users express behaviors as *if trigger then action* rules. IFTTT is a canonical example; a large-scale analysis of over 200,000 recipes revealed that users create diverse automation patterns across online services and devices [2]. Commercial platforms such as SmartThings, Home Assistant, and visual flow-based tools such as Node-RED similarly expose sensors, actuators, and services as composable blocks, lowering the barrier to IoT automation while binding rules to platform- or vendor-specific device identifiers [4, 10, 15, 16]. This tight coupling limits portability when devices are replaced or ecosystems change.

To improve portability, ontology-based approaches, such as EUPont, abstract device capabilities into high-level semantic concepts, enabling rules to be written over properties (e.g., indoor temperature) rather than concrete device IDs [9]. Task-oriented IoT languages, such as mTask, further compile high-level tasks into device-agnostic bytecode deployable across heterogeneous hardware [5]. These systems motivate the use of device abstractions but still assume that automation logic is authored directly in a structured language rather than being derived from natural language.

A complementary line of work explores conversational or NL-based interfaces for IoT applications. HeyTAP guides users through conversational queries to derive IF-THEN rules [17], whereas InstructableCrowd crowdsources TAP rules from user-described problems [3]. More recent systems, such as ChatIoT, leverage large language models to synthesize TAP-style rules from conversational instructions [18]. These systems demonstrate the usability benefits of natural language interfaces but largely inherit the expressiveness limits and brittleness of TAP-style representations.

Beyond TAP, Zhang et al. translated natural language smart-home requirements into Linear Temporal Logic (LTL) specifications that can be formally verified, highlighting the importance of temporal expressiveness and formal semantics in IoT automation [19]. This line of work motivates the need for DSLs that expose temporal and stateful constructs while remaining accessible to end users.

2.2. Natural Language to Other DSLs

Our work is also related to broader efforts to transform natural language into executable domain-specific languages. In text-to-SQL, early sequence-to-sequence models frequently produce syntactically invalid queries. PICARD addresses this issue by incrementally parsing partially generated outputs and rejecting tokens that violate SQL grammar or schema, substantially reducing syntactic errors [20]. This parser-in-the-loop strategy exemplifies how grammar-level constraints can be enforced during the decoding process. In contrast, the present work assumes cloud-hosted LLM APIs where token-level intervention is not consistently available. Reliability is therefore addressed through service-aware context construction, post-generation deterministic validation, and repair. In addition, text-to-SQL and IoT DSL generation differ in an important way: beyond grammatical well-formedness, IoT automation requires service-schema grounding, temporal execution semantics, and policy constraints tied to physical devices.

Task-oriented DSLs for IoT, such as mTask [5], occupy a complementary position by providing high-level abstractions for distributed applications that are compiled into platform-independent representation. However, these systems target expert developers and do not address NL-driven program synthesis.

In the robotics domain, several systems translate NL instructions into plans and executable programs. PaLM-SayCan decomposes high-level instructions into sequences of low-level skills filtered by affordance models [21], whereas Code-as-Policies generates Python programs that invoke robot APIs through hierarchical prompting [22]. These approaches illustrate the effectiveness of combining LLM reasoning with domain-specific action libraries and feasibility checks; however, they target embodied agents rather than heterogeneous IoT environments with complex temporal and policy constraints.

Table 1

Comparison of representative prior work grouped by reference sets and capability axes. Each column aggregates a line of related research discussed in Section 2.

	TAP platforms & ontologies	NL interfaces for IoT	NL→DSL text-to-SQL	Constraint- guided & self-repair	Edge-cloud IoT runtimes	Proposed pipeline
Natural-language front-end	✓	✓	✓	✓	–	✓
Device / service abstraction	limited	partial	–	–	✓	✓
Temporal and stateful expressiveness	limited	limited	limited	partial	✓	✓
Grammar / schema validation	–	–	✓	✓	–	✓
Safety and policy constraints	limited	partial	partial	partial	✓	✓
Static checking / verification	partial	✓	partial	✓	–	✓
Self-repair and RAG-based reasoning	–	–	–	✓	–	✓
Runtime execution awareness	–	–	–	–	✓	✓

Representative references: TAP platforms and ontologies [2]; NL interfaces for IoT [18]; NL→DSL text-to-SQL [20]; constraint-guided and self-repair methods [26]; edge-cloud IoT runtimes [34]. ✓: explicitly supported; partial: indirectly or partially supported; limited: restricted expressiveness; –: not a design goal.

More generally, declarative query languages for LLMs, such as LMQL, provide structured control over the decoding and execution flow [23]. Although not specific to IoT, they demonstrate how an intermediate language can mediate between natural language and constrained execution.

2.3. LLM-Based and Constraint-Guided Code Generation

Large language models trained on source code, such as Codex and Code Llama, have shown strong performance in NL-conditioned code generation [11, 24]. However, unconstrained generation often yields syntactic or semantic errors in domain-specific settings, and has been shown to produce insecure code in security-critical contexts [25]. This has motivated extensive research on constraint-guided decoding. JSONSchemaBench systematically evaluates schema-guided generation, showing that structural constraints can improve both output validity and efficiency [26]. Grammar-guided decoding frameworks similarly mask invalid tokens using formal grammars or regular expressions [27–29].

Beyond syntactic constraints, retrieval-augmented generation (RAG) [30] and chained reasoning have emerged as effective tools for improving factual grounding and logical coherence in LLMs. CoT-RAG combines chain-of-thought prompting with retrieval, allowing models to reason step-by-step while consulting external knowledge sources [31]. Separately, self-refinement and self-correction mechanisms enable models to revise outputs based on feedback or detected violations [12, 32, 33]. These techniques treat code generation as a closed-loop process, rather than a single-pass prediction.

2.4. Edge Cloud Architectures for IoT

Another line of research focuses on runtime architectures for IoT applications. Hierarchical and service-oriented designs push computation towards the edge to reduce latency and improve privacy while maintaining cloud connectivity for coordination and management [34]. Industrial platforms such as AWS IoT Greengrass and Azure IoT Edge provide container-based runtimes for deploying application logic and ML models on edge gateways [35, 36]. These systems address deployment and execution concerns but assume that the application logic is manually authored.

Taken together, prior work addresses important components of natural-language-driven automation, including TAP authoring, device abstraction, NL-to-DSL generation, constraint-guided decoding, and edge-cloud execution. The gap is that these capabilities are rarely integrated into a single IoT-oriented pipeline that combines service-level abstraction, temporal/stateful representation, LLM-assisted generation, deterministic validation, repair, and runtime-aware execution constraints. Table 1 summarizes this positioning; the final column refers to the proposed NL-to-JOILang pipeline rather than to JOILang alone. The next section introduces JOILang, the service-level representation on which the proposed pipeline is built.

3. JOILang as a Service-Level IoT Representation

This section introduces JOILang, a DSL designed to control IoT environments using natural language descriptions. We first analyze why controlling IoT systems via natural language is inherently challenging and then describe the syntax and execution semantics of JOILang, explaining how its design addresses these challenges.

3.1. Natural-Language Ambiguities in IoT Control

User-issued natural language commands are inherently *compressed and underspecified representations* of the precise control specifications required by IoT systems. A single utterance may admit multiple interpretations along several dimensions, including device scope, temporal and event semantics, condition–action structure, and device-level conceptual ambiguity. If these ambiguities are not properly resolved, the resulting automation scenarios may significantly diverge from the user’s true intent. We group the primary sources of structural ambiguity in natural-language IoT commands into the following five categories.

- **Device scope specification:** Commands such as “switch on the lights,” which lack an explicit *target device set*, are ambiguous as to whether they refer to all lights in a given space, a specific subset, or an arbitrary single device.
- **Temporal and event semantics:** Expressions such as “when the door closes” and “while the door is closed” are linguistically similar but semantically distinct. The former denotes a *transition-based event* triggered by a state change, whereas the latter represents a *static predicate* in the current system state.
- **Reference time and scheduling:** Natural-language commands often omit repetition frequency, termination conditions, or temporal reference points. For example, the command “switch off the pump three seconds after the light switches off, and close the blinds five seconds later” is ambiguous as to whether both delays are measured from the light-off event or whether the second delay is relative to the pump shutdown.
- **Condition–action structure:** When multiple conditions and actions are specified together, for example, “if it is hot, switch on the air conditioner; if the humidity is also high, set the dehumidifier to auto mode,” it is often unclear (i) which conditions apply to which actions, (ii) the scope of each conditional block, and (iii) whether the actions are intended to execute sequentially or conceptually in parallel.
- **Device-level ambiguity:** Natural-language expressions such as “shade” and “blind,” or “alarm” and “siren,” are frequently used interchangeably by users, despite corresponding to distinct device types and service capabilities in real IoT deployments. Ambiguity increases further when multiple devices with similar functions coexist in the same environment. This device-level synonymy motivates our use of alias-based normalization and recall-oriented multi-candidate retrieval, rather than exact single-label matching, in the pre-mapping stage.

3.2. JOILang Scripts and Execution Semantics

As discussed above, natural language commands are ambiguous along multiple dimensions, including device scope, temporal semantics, and condition–action structure. JOILang is a domain-specific scripting language designed to resolve these ambiguities while remaining suitable for both automatic translation from natural language and direct authoring by users.

Rather than requiring users to manipulate individual devices, JOILang allows IoT applications to be specified at the level of abstract *services*. Its syntax is intentionally lightweight, and it introduces a tag-based device abstraction mechanism that enables users to declaratively specify device scopes. In addition, the language restricts the control-flow constructs to facilitate static analysis and policy verification.

3.2.1. Program Units and Lifecycle: *cron* and *period*

In JOILang, an IoT application is represented as a *scenario*, which constitutes the smallest executable unit in the program. Each scenario is defined using the following JSON structure:

```
{
  // scenario name
  "name": string,
  // CRON expression specifying scenario start times
  "cron": string,
```

```
// execution period in milliseconds
"period": integer,
// JOILang script body
"code": "...
}
```

Unlike general-purpose programming languages, JOILang does not explicitly express looping constructs within the script body. Instead, repetition is handled by the combination of *cron* and *period*, which significantly simplifies scripting syntax. This design allows nested temporal patterns, such as “every hour on Mondays,” to be expressed in a structured and analyzable form.

The *cron* field follows the standard UNIX five-field format (*minute, hour, day-of-month, month, day-of-week*) and specifies when a scenario is initiated. For example, "0 0 * * *" denotes midnight every day, "* / 5 * * * *" denotes every five minutes, and "0 9 * * 1-5" denotes 9 a.m. on weekdays. An empty string "" is used as the default, indicating that the scenario starts immediately and is not subsequently restarted by the cron schedule.

The *period* parameter determines how often the code block is executed after the scenario starts.

- *period* = -1: the script is executed once and the scenario then terminates.
- *period* = 0: the script is executed repeatedly using the default interval of 100 ms.
- *period* >= 100: the script is executed repeatedly at the explicitly specified millisecond interval.

If a *break* statement is executed within the script, the current *period* loop terminates early, and no further execution occurs within the same cron cycle. The entire scenario restarts when the cron expression matches again. Each scenario permits exactly one *period*; parallel loops must be expressed using multiple scenarios. When a natural language command does not explicitly specify repetition, the default configuration *cron* : "" and *period* : -1 is applied.

3.2.2. Device Abstraction

In JOILang, IoT devices are abstracted as either *value services*, which expose sensor readings, or *function services*, which perform actions. Scripts are written using a predefined registry of such services rather than using direct device identifiers.

To distinguish between device types, locations, and roles, JOILang introduces hash-based tags. Tags may represent device classes, installation locations, or functional roles, and multiple tags denote the intersection of their corresponding device sets. For example, (#living #Light) refers to light-type devices located in the living room. When multiple devices match a tag set, the optional quantifiers, *any* and *all*, specify whether an action applies to an arbitrary single device or all matching devices. If omitted, JOILang conservatively interprets the target as a single arbitrary device from the intersection.

Traditional trigger-action rules can be expressed concisely at the service level. For example, the command “if the door is closed, sound the alarm once” is written as:

```
if ((#Door).doorControl_door == 'closed')
  (#Alarm).alarm_siren()
```

Here, *doorControl_door* and *alarm_siren* are registered value and function services, respectively, while #Door and #Alarm are device tags. This abstraction allows JOILang to resolve the device-level ambiguity inherent in natural language.

3.2.3. Variables

JOILang supports variables within scripts, primarily to store sensor values. The variable lifetime is controlled by two distinct assignment operators:

- *a* := *v*: declares and initializes a variable when a cron-triggered scenario begins. The variable state is shared across the *period* iterations of the same cron cycle, and an updated value remains available to later iterations; when a new cron cycle begins, the variable is initialized again.
- *a* = *v*: assigns or reassigns a value whenever the statement is executed. Within a periodic body, the right-hand side is evaluated again on each iteration, and the operator can also update variables declared with :=.

For example, the command “switch on the light only when it becomes darker than the previous measurement” can be expressed as:

```
p_brightness := 1000.0
c_brightness =
  (#LightSensor #living).lightLevel_light
if (c_brightness < p_brightness) {
  (#Light #living).switch_on()
}
p_brightness = c_brightness
```

3.2.4. Control Structures: *if and wait until*

Dynamic execution requires conditional and temporal control constructs. As discussed above, most repetition patterns are captured using `cron` and `period`. To distinguish between static conditions and transition-triggered execution, JOILang provides two complementary constructs:

- Static state branching: `if, else if, else`.
- Blocking execution until a condition becomes true: `wait until(condition)`.

Conditions may include comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`), logical operators (`and`, `or`), and parentheses. For example, “switch off the light if the door is closed” is expressed using a simple `if` statement, whereas “sound the alarm once when the door closes” is written as:

```
wait until ((#Door).doorControl_door == 'closed')
  (#Alarm).alarm_siren()
```

Commands that require repeated transitions, such as “sound the alarm every time the door opens,” are expressed using a periodic loop (`period = 0` or `period >= 100`) combined with an explicit state comparison.

```
{
  "name": "Scenario1",
  "cron": "",
  "period": 100,
  "code": "
    prev := 'closed'
    state = (#DoorLock).doorControl_door
    if ((state == 'open') and (prev == 'closed')) {
      (#Alarm).alarm_siren()
    }
    prev = state
  "
}
```

When quantifiers appear in conditions, they are interpreted as follows:

- **any**: the condition holds if at least one matching device satisfies it.
- **all**: the condition holds only if all matching devices satisfy it.

In summary, JOILang is designed around abstract services and spatial/functional tags rather than individual devices. By resolving concrete device sets at runtime through tag-based mapping, it loosely couples automation scripts with physical devices, enabling scalable and vendor-agnostic IoT automation in large deployments [13]. Moreover, JOILang adopts conservative default interpretations to handle natural language ambiguity and supports iterative correction through user feedback when misinterpretations occur. The next section describes how the proposed pipeline reliably generates such scripts from natural-language commands.

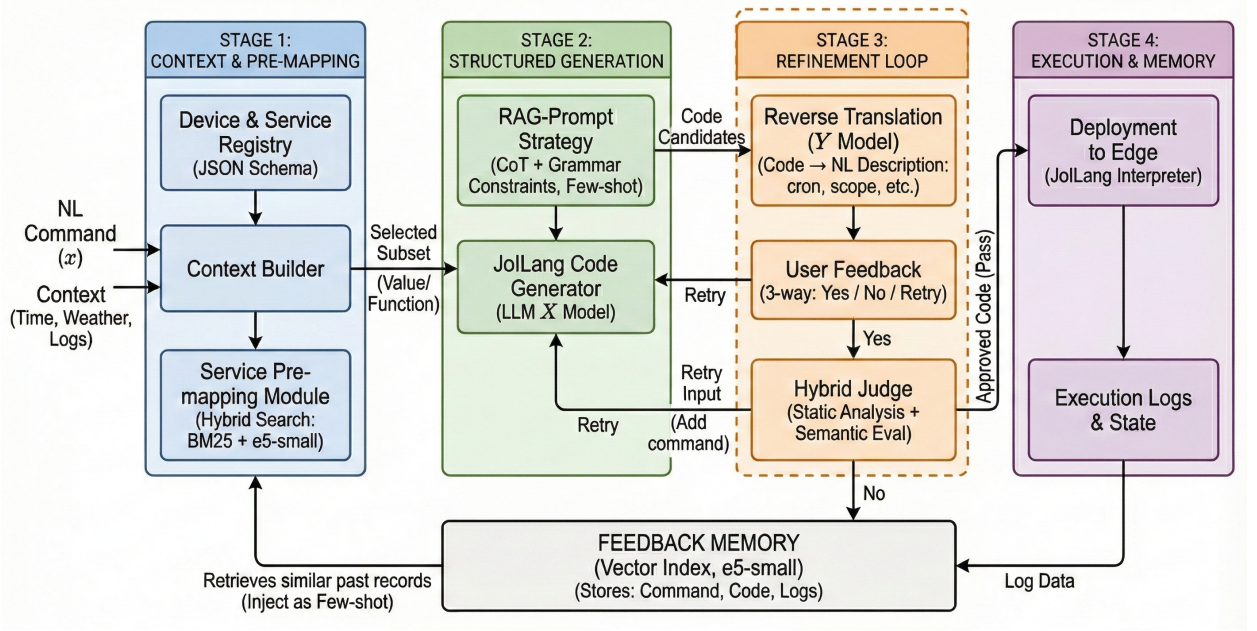


Figure 2: End-to-end NL-to-JOILang pipeline for service-context construction, generation, validation, repair, and deployment.

4. Reliability-Oriented NL-to-JOILang Pipeline

This section describes the proposed NL-to-JOILang pipeline. In this paper, the term *framework* refers only to the complete pipeline in Fig. 2: natural-language input, service-context construction, JOILang generation, deterministic validation, repair, user-facing explanation, and evaluation. It does not refer to JOILang alone, the service abstraction alone, or the runtime resolver alone. The pipeline is designed to improve the correctness of generated scripts while controlling prompt length, generation latency, and the cost of using cloud-hosted LLMs. Based on these objectives, we designed the pipeline shown in Fig. 2, which consists of four stages.

In the first stage, the translator takes the user’s natural language command and available context as input and selects a subset of services that are likely to be used in the script. This stage maintains a JSON schema of registered devices and their services and uses it, together with contextual information (e.g., current time and weather), to perform service pre-mapping (Section 4.1), identifying candidate device sets that can perform the requested behavior.

The second stage generates a JOILang script by combining RAG-style retrieval and prompt engineering. The third stage refines the generated script by reverse translation: the script is translated back into natural language and presented to the user to verify whether it matches the original intent. If the behavior is not aligned, the user may express additional requirements in natural language; these are fed back into the second stage to regenerate or patch the script. Even when the user accepts a candidate, the system still checks whether the script satisfies JOILang grammar and policy constraints before its deployment.

Finally, in the fourth stage, scripts that pass both semantic and static checks are deployed to the runtime, where they are executed on the actual IoT system. The following subsections describe the core techniques used at each stage.

4.1. Service Pre-mapping

For robust IoT control, it is essential to clearly distinguish between *what state to read (VALUE)* and *what actions to perform (FUNCTION)*. If this distinction is blurred, the model may generate syntactically valid code that nevertheless references non-existent devices or services, or that is triggered under incorrect conditions, yielding *non-functional* or *unsafe* code.

A key difficulty in implementing the NL-to-JOILang translation stage is that a general-purpose LLM has no built-in knowledge of the services registered in the JOILang catalog. As the number and variety of IoT devices increase, the amount of service information that must be exposed to the LLM increases proportionally. Our earliest prototype

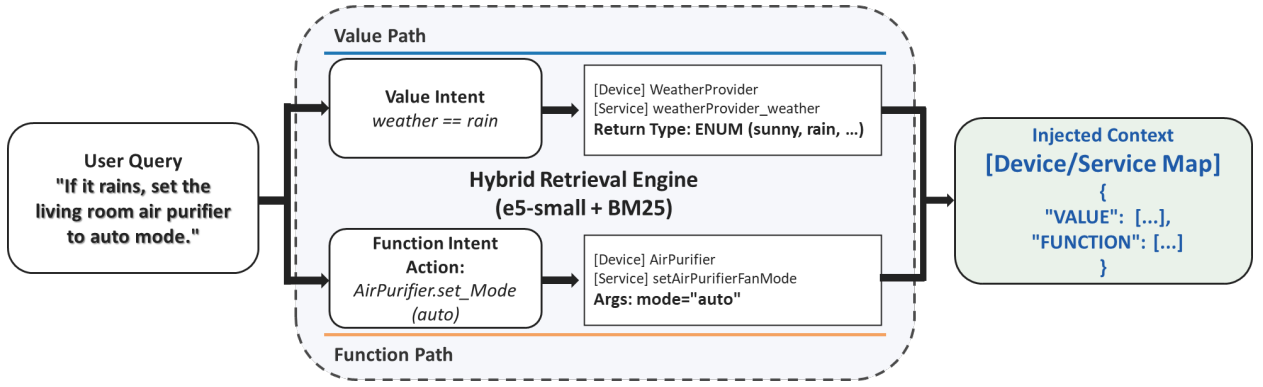


Figure 3: Service Pre-mapping overview. The natural language command is first split into VALUE (condition) and FUNCTION (action) buckets, then each bucket is processed by a lightweight hybrid retriever (e5-small + BM25) to select a minimal subset of relevant services, which is then injected as the [Device and Service Mapping] context for JOILang code generation.

exposed the device registry to the model in its native form, the Python class definitions used by the runtime, which alone amount to approximately 29K tokens. Restructuring the registry into a flat JSON schema of devices, services, types, enumerated values, and numeric ranges reduced the catalog to roughly 21K tokens, removed the Python class and decorator boilerplate, and provided a uniform representation for the subsequent retrieval and generation stages; this JSON catalog is the schema that every later stage of the pipeline consumes. Even in this compact form, combining the catalog with a system prompt that includes JOILang grammar, safety rules, usage examples, and evaluation criteria produces a context that is extremely large and difficult for the LLM to use effectively. In practice, we observed that such “full catalog” prompts cause the model to hallucinate device–service mappings or invent entirely non-existent services, consistent with recent evidence that LLMs struggle with reliable API discovery and service ranking in edge AI and IoT settings [37]. Therefore, this approach is impractical in terms of both reliability and token budget. To address this problem, we introduce a preprocessing stage called *Service Pre-mapping*.

4.1.1. Methodology: Intent Decomposition and Hybrid Retrieval

The core idea is to first decompose the natural language command into *conditions* (VALUE) and *actions* (FUNCTION) (Fig. 3), and then apply a lightweight hybrid retrieval model (e5-small [38] + BM25 [39]) to each bucket to select only those VALUE/FUNCTION services that are likely to be required. The resulting *minimal subset* is fed into the JOILang generator as the [Device and Service Mapping] block in the prompt. Compared to injecting the full catalog, this drastically reduces the token footprint while encouraging the decoder to form consistent condition–action combinations from a restricted candidate pool.

In the Service Pre-mapping stage, the natural language command is interpreted and split into parts corresponding to VALUE services (state evaluation) and FUNCTION services (device actions). For example, in “If it rains, switch the living-room air purifier to auto mode,” the phrase “if it rains” is mapped to VALUE services, while “switch the air purifier to auto mode” is mapped to FUNCTION services. Each of these subqueries is then used in a natural language retrieval step to select candidate services from the JSON schema.

We use a hybrid retriever that combines e5-small embeddings with a lightweight BM25-based lexical signal. Embedding-based similarity remains the primary driver of retrieval performance, while BM25 is only used to mitigate edge cases involving short or ambiguous service identifiers. In isolation, lexical matching is insufficient for our task; however, when used as a secondary signal, it contributes to improved robustness in the IoT domain.

First, we enforce hard schema constraints by masking out candidates that violate required type or signature conditions, assigning them a score of zero. Next, for a query q and a service description d , we compute the combined score

$$s(q, d) = w \cdot \cos(e_q, e_d) + (1 - w) \cdot \text{bm25}(q, d) \quad (1)$$

where $\cos(e_q, e_d)$ is the cosine similarity between the corresponding e5-small embeddings and $\text{bm25}(q, d)$ is the BM25 score; both terms are min–max normalized before combination.

Table 2

Representative embedding candidates for JOILang service pre-mapping (English setting). We report nDCG@10 and Recall@10 on our IoT retrieval benchmark (higher is better) and mean end-to-end retrieval latency per query on an NVIDIA A6000 server (lower is better). The table compares hybrid and dense-only retrievers and also isolates the effect of VALUE/FUNCTION splitting under the same intfloat/e5-small + BM25 backbone.

Model	nDCG@10	Recall@10	Latency (ms)	Search
intfloat/e5-small + BM25 (split)	0.9830	1.0000	9.1	hybrid
intfloat/e5-small + BM25 (no split)	0.9809	0.9981	7.8	hybrid
mixedbread-ai/mxbai-embed-xsmall-v1 + BM25 (split)	0.9798	1.0000	8.0	hybrid
intfloat/multilingual-e5-small (split)	0.9783	0.9996	12.5	dense-only
BAAI/bge-m3 (split)	0.9807	0.9996	24.4	dense-only

Note that parameter w is not merely an empirical hyperparameter, but a safety-critical design choice: we may conservatively lower w to favor lexical and structural evidence over dense semantic similarity so that non-existent or type-incompatible IoT services are strongly penalized during retrieval. An `aliases` field is used to handle multilingual synonyms without increasing the retrieval latency.

The top- k_{device} candidates from the VALUE and FUNCTION buckets are then deduplicated and reduced to $N_{\text{service}} \leq 12$ services, while keeping the VALUE and FUNCTION candidates explicitly separated. Each entry includes its name, type, ENUM or numeric range, description, and aliases; the injected shortlist comes to roughly 0.9K tokens in place of the 21K-token catalog. This role-separated subset becomes the [Device and Service Mapping] block in the JOILang generation prompt. Potential errors (e.g., forbidden devices or unsupported types) are screened by a *static gate*, and if necessary, the pre-mapping step is repeated with adjusted parameters.

Service pre-mapping is not a hard single-choice decision about the final physical device. It is a recall-oriented, multi-candidate grounding step that narrows the schema context while preserving semantically related services for downstream disambiguation. In practice, we use a recall-first thresholding policy together with a bounded top- k_{device} candidate set so that ambiguous or non-standard phrasing is less likely to eliminate the correct service too early. Final device binding is resolved later through JOILang’s tag-based late binding at runtime, and residual ambiguity can be handled through deterministic validation and the reverse-translation feedback loop.

4.1.2. Preliminary Experiments: Exploring Retrieval Structures

The key component of Service Pre-mapping is the retrieval model that, given a natural language query, selects relevant services from a JSON-based catalog. In general, natural language retrieval methods can be grouped into three categories: keyword-based methods, such as BM25; dense embedding methods, such as e5-small; and hybrid methods that combine both.

We conducted preliminary experiments with more than 20 embedding and retriever combinations suitable for our system, including two lightweight hybrid configurations (intfloat/e5-small + BM25 and mixedbread-ai/mxbai-embed-xsmall-v1 + BM25 [40]), and compared them against dense-only embedding models such as intfloat/multilingual-e5-small [41] and BAAI/bge-m3 [42]. Representative English-setting results are summarized in Table 2. Corpus documents are description-only role and merged documents built from the JSON catalog and its device aliases (84 and 53 documents; no example command appears in any document). The evaluation covers 1,200 held-out queries in five pools: template, natural, and unseen-paraphrase single-device commands, plus compound commands that involve two or three devices (e.g., “If the light is on, turn on the air conditioner”). The gold label of every query is the set of devices it requires, credited through acceptable-alternative groups (a light and a light sensor both count for a light-state condition), and context-dependent fragments that carry no device evidence in their own text are excluded. The e5-small encoder of the adopted configuration is fine-tuned on 2,305 held-in commands spanning all three registers (MultipleNegativesRankingLoss, ten epochs, batch 32, seed 2025); the no-split ablation shares this encoder and the same training queries, differing only in the document granularity, and the other retrievers are used as released. Every configuration retrieves $k_{\text{device}} = 10$ candidate devices at the default retrieval weight $w = 0.6$, and no evaluation query is seen during training.

As shown in Table 2, the results highlight three consistent trends.

First, VALUE/FUNCTION splitting outperforms the no-split configuration on every metric under the same fine-tuned e5-small + BM25 backbone (nDCG@10 of 0.9830 versus 0.9809, Recall@10 of 1.0000 versus 0.9981), and the gap is concentrated exactly where it matters: on compound commands the no-split ranking falls to an nDCG@10 of 0.9728 while the split configuration keeps 0.9932. The reason is structural. The condition clause of a command is searched against the VALUE bucket and the action clauses against the FUNCTION bucket, so each device involved in a multi-device command surfaces in its own role bucket instead of competing in one shared ranking; for “If the light is on, turn on the air conditioner,” the condition clause retrieves the light as a VALUE candidate while the action clause retrieves the air conditioner as a FUNCTION candidate. The role-labeled shortlists are then injected as explicitly separated subsets of the [Device and Service Mapping] block, so the decoder can map conditions to VALUE services and actions to FUNCTION services directly during JOILang generation.

Second, the Recall@10 of 1.0000 for the adopted configuration is the property this stage is designed around. Service Pre-mapping exists to shrink the catalog context, not to make the final device decision, and once the 21K-token catalog is out of the prompt it makes almost no difference to the token budget whether the shortlist keeps five candidates or ten. The candidate budget is therefore spent on recall: a ten-device list that never loses a required device, including compound commands that span two or three devices, lets the runtime shortlist (top- k_{device} per role, $N_{\text{service}} \leq 12$ services) narrow the catalog at no cost to the downstream generator.

Third, the adopted configuration achieves the best ranking quality at practical cost. Larger dense models stay behind (BAAI/bge-m3 reaches an nDCG@10 of 0.9807 and is also the slowest at 24.4 ms per query), so a lightweight encoder adapted to the catalog, combined with the lexical anchor of BM25, remains the better fit; the same configuration also came out ahead in the broader sweep of more than 20 embedding and retriever combinations described above.

While dense-only models remain attractive in broader multilingual settings, our IoT catalog is dominated by a relatively small vocabulary of short keywords and ENUM-like expressions. In this context, larger dense-only models trail the adopted configuration on every metric while the largest is also the slowest, so they do not provide quality gains that would justify displacing the lightweight domain-adapted hybrid retriever. These findings align with the design goal of Service Pre-mapping: rather than maximizing generic semantic similarity, the retriever must reliably narrow the schema context while minimizing cross-role confusion and type-incompatible candidates.

Accordingly, we adopt `intfloat/e5-small + BM25 (split)`, with the domain-adapted encoder, as the default configuration in all downstream experiments. The resulting minimal subset is injected into the JOILang prompt as the [Device and Service Mapping] block, in which the VALUE and FUNCTION candidates remain explicitly separated, constraining the decoder to construct condition–action pairs strictly from this filtered, role-labeled candidate pool. In later sections, we show that this high-recall pre-mapping contributes directly to improved reliability, safety, and end-to-end NL-to-DSL generation performance.

4.2. Prompting Strategies and RAG-Prompt Mechanism

Within the reduced search space enabled by Service Pre-mapping, the second stage synthesizes DSL code that satisfies the detailed temporal and state-transition constraints of JOILang. This requires careful design of context construction (the *context composition strategy*) and model reasoning (the *structured reasoning strategy*). In this subsection, we define the five prompting strategies evaluated in Section 5 and describe our proposed Pre-map–CoT+RAG method, which combines Service Pre-mapping, RAG-based prompt block selection, and chain-of-thought (CoT) reasoning augmented with state-aware skeleton generation for temporally complex commands (Section 4.2.2).

The main strategies we explore can be characterized as combinations of (i) context construction and (ii) reasoning style. All prompt templates followed the same basic structure in our open-source implementation.

- **Baseline–Full Prompt:** Injects the entire JOILang manual and the full service catalog directly into the prompt, with no structure-aware selection, and then asks the model to generate the code.
- **Baseline–CoT:** Uses the same full prompt as above, but instructs the model to perform CoT reasoning [43] by first writing a step-by-step explanation in natural language before producing the JOILang code.
- **Pre-map–Full Prompt:** Applies Service Pre-mapping to select VALUE/FUNCTION candidates but still injects the full grammar section and other documentation without RAG-based selection.
- **Pre-map–CoT+RAG (Proposed):** Our proposed method combines Service Pre-mapping, RAG-based prompt block selection and injection, and CoT-based structured reasoning with a state-aware skeleton.

- **Pre-map–CoD**: Uses Service Pre-mapping and a **Chain-of-Draft (CoD)** strategy [44] in which the model first produces a compressed reasoning draft in a few keywords per step, and then expands it into JOILang code.

We refer to Baseline–CoT and Pre-map–CoT+RAG collectively as the CoT family, as both employ CoT-style stepwise reasoning. In Pre-map–CoT+RAG, we further incorporate *State-Aware Skeleton Generation* within the CoT process to handle commands with complex temporal constraints, adapting the skeleton-of-thought idea [45] to explicit state-based planning. Importantly, skeleton generation is not treated as a separate prompting strategy, but as a constrained and structured instantiation of CoT that enforces explicit state transitions. Accordingly, references to “CoT” in Table 5 include skeleton-augmented CoT.

Below, we first describe the *RAG-Prompt* mechanism for selecting grammar and rule blocks, outline the structured reasoning with the state-aware skeleton, and compare the CoT family to CoD.

4.2.1. The RAG-Prompt Mechanism (Context Optimization)

Atomic Prompt Blocks and Query Router If the entire JOILang manual is injected as a single block of text, much of it will be irrelevant to any given command, and the key rules may be diluted, leading the model to ignore the critical constraints. To avoid such context dilution, we decompose the grammar rules, safety policies, and representative usage patterns into 22 *atomic prompt blocks*. Each block covers a single concept and a small number of examples, such as *Temporal Rules*, *Loop Constraints*, *Condition Combination Rules*, *Type & Boundary Constraints*, and *Any/All Quantifier Patterns*. These blocks are stored as Markdown files under the `prompts/` directory.

Given an input command, a *Query Router* first tags the command into normalized categories. We use four categories:

- [Loop - Cron]: Time-based triggers that define when the scenario starts (e.g., “every day at 7 a.m.,” “on weekdays only”).
- [Loop - Period]: Expressions indicating periodic monitoring or repetition (e.g., “every 3 minutes,” “keep watching”).
- [Condition]: Logical conditions on state, thresholds, or events (e.g., “if the lights stay on,” “if the temperature exceeds 30°C”).
- [Nested Condition]: Nested structures where conditions evolve over time (e.g., “if the room is empty and then someone enters”).

The Query Router segments the command into semantic chunks and assigns each chunk to one or more categories. For instance, “Every day between 6 p.m. and 7 p.m., open the windows for 15 minutes and then close the windows” involves [Loop - Cron] and [Loop - Period] but no conditions, so only prompt blocks related to temporal rules and loop constraints are selected; condition-combination blocks are skipped. In contrast, a command such as “If the lights stay on for 30 minutes, send a notification” involves both [Loop - Period] and [Condition] and requires consulting the condition-combination rules, which classify the pattern into one of several types (static conditions, transitions, mixed state and transitions, one-shot triggers with repeated actions, or composite conditions). Commands without explicit conditions bypass this classification to avoid unnecessary reasoning.

Hybrid RAG Retrieval and Context-Aware Assembly Each prompt block is indexed with `intfloat/e5-small` embeddings and a BM25 index. For a given command, we retrieve the top- k_{prompt} blocks, ranked by hybrid similarity. Unlike a fixed- k_{prompt} design, we set k_{prompt} *adaptively* based on the *length and complexity of the input command*: shorter commands typically require only a small number of blocks (e.g., k_{prompt} near 3), whereas longer commands with richer temporal/conditional structures benefit from retrieving more blocks (i.e., larger k_{prompt}). This yields a command-aware context size without enforcing a hard token budget upper bound as the primary control mechanism.

These selected blocks are concatenated with the Service Pre-mapping result, which is formatted as the [Device and Service Mapping] section, and together they form the system prompt. Specifically, the RAG-Prompt assembles the context in the following order:

1. A **core block** containing essential JOILang syntax and safety rules.
2. The subset of **atomic prompt blocks** selected via RAG (e.g., temporal rules, loop constraints, condition-combination rules) based on the Query Router’s tags.

3. The **VALUE/FUNCTION service subset** from Service Pre-mapping, as the [Device and Service Mapping] section.
4. The JSON output format specification for scenarios (fields name, cron, period, code).

Since prompt blocks are cumulatively stacked on top of the core block, as command complexity increases, the triggered rule blocks accumulate toward a near-full context rather than a fragmented partial context. This design dramatically reduces the number of input tokens relative to the full prompt while preserving all grammar and service information relevant to the current command, thereby improving the *context efficiency*. The lower average token count and latency of the Pre-map-CoT+RAG strategy in Table 5 reflect the combined effect of RAG-Prompt and service pre-mapping.

4.2.2. Structured Reasoning: CoT Family with State-Aware Skeleton Generation

CoT Family and the Role of State-Aware Skeleton Generation Both Baseline-CoT and Pre-map-CoT+RAG instruct the model to produce an intermediate reasoning trace before emitting the JOILang code. The model first explains, step by step, “which states to observe, in what order, and under which conditions to perform which actions,” and only then translates this reasoning into code.

However, plain CoT is often insufficient for complex temporal logic involving multiple delays, state transitions, and accumulated time constraints; in such cases, the model may default to patterns such as chained `clock_delay` calls or blocking loops that conflict with JOILang’s design. To avoid this, Pre-map-CoT+RAG incorporates State-Aware Skeleton Generation as a constrained reasoning template for commands with non-trivial temporal structure, inspired by skeleton-of-thought prompting [45]. The model first designs a hierarchical state machine over triggers, waiting phases, and elapsed-time checks, and only then instantiates this structure into executable JOILang code. This terminology is intentionally chosen to emphasize reliability-oriented state planning rather than generic “thought” tracing.

Phase 1: Logic Decomposition and State Initialization In Phase 1, the command is decomposed into triggers, phases, and global variables. The model outlines which event triggers start the scenario (e.g., door open/close, time range, sensor threshold), how many logical phases are needed (Idle, Delay, Logic, Duration, Reset, etc.), and which global variables are required. Typical global variables include:

- `step := 0`: an index tracking the current phase or state.
- `triggered := false`: a flag to prevent unintended re-triggering.
- `startTime := 0`: the reference timestamp for elapsed-time calculations.
- `retry_count := 0`: a counter for repeated actions or retries.

The outcome of this phase is a natural language description of (i) the triggers, (ii) the phase structure, and (iii) the initialization of global variables in terms of JOILang’s `:=` and `=` semantics.

Phase 2: Hierarchical State-Machine Skeleton In Phase 2, the model designs a state-machine skeleton consisting of **Step 0, Step 1, Step 2, ...**, without yet using the actual JOILang syntax. Each step is described by the following four elements:

- **Type**: Either `Wait Until` (event-based blocking) or `If` (periodic checking).
- **Condition**: A conjunction of `step == N` with sensor states, elapsed time conditions, and global flags (e.g., `triggered`, `retry_count`).
- **Nested Logic**: The internal `if / else if / else` structure within the step (e.g., “if dark, use main light; otherwise, use mood light”).
- **Transition**: The next step index and any updates to `startTime`, flags or other variables.

To make this skeleton robust, we enforce three rules at the prompt level.

- **Rule 1 (No Blocking Loops):** Long waits must *not* be implemented with blocking constructs such as `while`, `sleep`, or `busy-wait`. Instead, the model must rely on periodic checks of conditions such as `if (step == 1 AND elapsed >= 5000ms)` within the period loop.
- **Rule 2 (Delays as Elapsed Time):** All delays must be expressed in terms of elapsed time, `CurrentTime - startTime`. The variable `startTime` is recorded in the trigger or previous step and reused in subsequent steps to determine when to transition.
- **Rule 3 (Nested Sub-conditions):** Branches such as “if dark, do X; otherwise, do Y” are represented as sub-conditions within the same step, each with its own actions and transitions.

For example, for a complex command such as “When the motion sensor detects movement, wait 3 seconds. If it is dark, the main light is switched on; otherwise, the mood light is switched on. Keep it on for 10 seconds and then switch everything off,” the state-aware skeleton might be as follows:

- **Step 0 (Trigger):**
Type = Wait Until,
Condition = `[value1] AND (triggered == false)`,
Action = `triggered = true, startTime = CurrentTime, step = 1`.
- **Step 1 (Delay):**
Type = If,
Condition = `step == 1 AND (CurrentTime - startTime) >= 3000ms`,
Action = `step = 2`.
- **Step 2 (Nested Logic):**
Type = If,
Condition = `step == 2`,
Sub-condition A = `([value2]) → [function call1]`,
Sub-condition B = `Else → [function call2]`,
Transition = `startTime = CurrentTime, step = 3`.
- **Step 3 (Duration & Reset):**
Type = If,
Condition = `step == 3 AND (CurrentTime - startTime) >= 10000ms`,
Action = `[function call1], triggered = false, step = 0`.

At this stage, the skeleton is expressed using abstract placeholders such as `[value]` and `[function call]` rather than device- or service-specific names. This abstraction captures the logical structure without prematurely committing to the JOILang syntax or mappings.

A similar construction handles commands such as “If no one enters within 5 minutes after the door opens, turn off the lights.” In this case, the trigger step records the door-opening event and initializes an elapsed-time reference. Subsequent periodic checks test two possibilities: whether motion is detected before the 5-minute deadline, in which case the pending action is cancelled or reset, and whether the elapsed time reaches the threshold without a new entry event, in which case the light-off action is executed. This example illustrates that the state-aware skeleton does not require a hand-authored finite-state machine for every utterance; rather, it externalizes dynamic temporal logic into explicit event flags, elapsed-time variables, and transitions that can later be instantiated in JOILang.

Phase 3: Syntax Instantiation into JOILang In Phase 3, the state-aware skeleton is instantiated as concrete JOILang code and wrapped into one or more JSON scenario objects. The global variables identified in Phase 1 are declared and initialized via `:=` at the top of the code block, and the period value is selected to match the monitoring frequency assumed in the skeleton. The Service Pre-mapping subset, provided as [Device and Service Mapping], is used to translate abstract references such as “Door == open” and “MainLight.on()” into the correct JOILang expressions, for example, `(#Door).doorControl_door == "open"` and `(#Light #Main).lightOperatingState_on()`. The final output is one or more JSON objects with the fields `name`, `cron`, `period`, and `code`.

Overall, Pre-map–CoT+RAG can be seen as a state-machine-oriented CoT: it uses Phases 1–3 to impose a structure on the reasoning process so that variables such as `step`, `startTime`, and `triggered`, as well as the nested conditions and transitions, are consistently aligned with JOILang’s semantics. In practice, this significantly reduces timeouts, mis-triggered events, and incomplete states, yielding the highest S_{DET} and pass rate among all strategies in Table 5.

4.2.3. Comparative Analysis: CoT Family vs. Chain-of-Draft (CoD)

Role and Limitations of CoD CoD [44] aims to reduce token usage by representing each reasoning step with only a few keywords. In our Pre-map–CoD baseline, the model first produces a compressed draft and then expands it into JOILang code. For example, the command “At 7 a.m., open the window and close it 10 minutes later” might yield the following draft:

```
[0] Cron 7am → [1] Window Open → [2] Wait 10m → [3] Window Close
```

Although this draft is concise, it omits crucial details such as the state variables (`step`, `triggered`, `startTime`), enumeration literals, and reset rules. As a result, the expansion to JOILang can easily introduce errors, such as incorrect ENUM values, missing resets, or unintended blocking loops. Empirically, Pre-map–CoD achieves competitive S_{DET} scores but lags behind Pre-map–CoT+RAG in terms of pass rate and error profile (Table 5). In our terminology, the drafts produced by CoD are treated as *keyword-based summaries*, not as full skeletons. The term “skeleton” is reserved for the hierarchical state-machine planning module described in Section 4.2.2.

Advantages of CoT with State-Aware Skeleton Generation (Pre-map–CoT+RAG) In contrast, the CoT family, and in particular Pre-map–CoT+RAG, does not attempt to compress the reasoning structure itself. Instead, it reduces token usage by optimizing the context via RAG-Prompt and Service Pre-mapping while preserving rich, structured reasoning through CoT and state-aware skeleton generation.

- **Context reduction via RAG:** Pre-map–CoT+RAG keeps the grammar and documentation compact by injecting only the relevant atomic prompt blocks and the filtered VALUE/FUNCTION subset, thus lowering the token count and latency without sacrificing crucial constraints.
- **State-machine-oriented CoT:** The state-aware skeleton imposes an explicit three-phase structure (state initialization, hierarchical skeleton, and syntax instantiation) that clarifies the roles of `step`, `startTime`, `trigger`, and nested conditions. This structure is particularly important for JOILang, where temporal logic and event-driven behavior are central concepts.

Our experiments show that in a DSL such as JOILang with strong temporal and state constraints, compressing reasoning via CoD is less effective than maintaining a well-structured CoT with state-aware skeleton generation combined with aggressive context pruning.

The Pre-map–CoT+RAG configuration offers the best balance between semantic correctness, deterministic admissibility, and efficiency in the evaluated benchmark.

4.3. Reverse Translation and Feedback

Even with Service Pre-mapping and grammar constraints, the generated code is still produced by an LLM and may deviate in subtle ways from the user’s intent. To mitigate this, our system combines a *reverse-translation-based three-way feedback loop* with a lightweight *feedback memory*.

For each candidate JOILang script, we invoke a separate “reverse translation” model (denoted M_Y) that explains in clear natural language what the code does. The prompt for M_Y is designed so that it explicitly mentions cron/period-based timing, any/all quantifiers, tag- and group-based scopes, and operations such as `wait until`, making the behavior easy to inspect for users.

For instance, suppose the user says “switch on the bulbs,” and the pre-mapping and generation steps produce `(#Light).switch_on()`. The reverse translation might describe this as “switches on one of the lights.” The user may then realize that the actual intent was “switch on all lights” and choose **Retry**, adding a clarification such as “Apply this to all bulbs.” The system then regenerates the script under the same schema and constraints, potentially yielding `all(#Light).switch_on()` as the new candidate. For each candidate, the user can provide one of three feedback signals:

- **Yes:** Accept the candidate and proceed to deployment.
- **No:** Reject the candidate and stop without deployment.
- **Retry:** Provide additional constraints or clarifications (e.g., “apply to all lights,” “repeat every 5 minutes”) in natural language; the system then regenerates the code using the existing Pre-mapping result and grammar constraints.

When the user ultimately accepts a script (Yes) and the scenario is deployed, the full interaction is stored as a *feedback memory record*, consisting of (1) the original command, (2) the reverse-translated description, and (3) the final JOILang code. These records are indexed using embeddings and stored in a vector database. When a new command arrives, the system retrieves similar past records and uses them as few-shot examples or prior candidates, thereby reducing the amount of user intervention and LLM computation required for recurring patterns over time, similar in spirit to caching-based reuse in LLM-based command-driven IoT systems [46].

Natural language commands from non-expert users may remain ambiguous even when the generation pipeline itself is structurally reliable. In this work, the reverse-translation loop is therefore positioned as a user-side inspection and correction mechanism that helps expose intent mismatches and reuse accepted corrections over time, rather than as evidence of validated user satisfaction or usability. Broader deployment-time studies on end-user trust, comfort, and interaction quality remain future work. The next section evaluates the proposed pipeline on JOICommands-170.

5. Experiments

This section evaluates the proposed NL-to-JOILang pipeline. We first measure end-to-end performance in terms of pass rate, deterministic quality S_{DET} , input tokens, and latency. We then report (i) representative-category component ablations, (ii) an ablation over the representation of the injected device/service context for the best-performing strategy, (iii) a sensitivity analysis for the retrieval weight w , and (iv) a clarification of the evaluation scope and the deployment-oriented configuration of the current experimental setting.

5.1. Experimental Setup

Dataset. We conducted all experiments on JOICommands-170¹. Each data point consists of (1) an English natural language command and (2) one or more JOILang reference scripts that satisfy the command. Each data point adheres to the schema summarized in Table 3. To facilitate difficulty-aware evaluation, the dataset is stratified by the *category* field, which classifies commands into 9 distinct types based on token length and logical complexity. The dataset contains 20 examples for each of the first eight categories (IDs 1–8), while the most advanced category (ID 9, Context-Aware) comprises 10 examples. These types are hierarchical, grouped into *Simple Control* (Level 1), *Conditional Control* (Level 2), and *Advanced Automation* (Level 3); the detailed definitions for each category are provided in Table 4. In the experiments reported in this paper, we use the English commands for all evaluations.

The dataset is used purely for evaluation. Model parameters were not fine-tuned on JOICommands-170, and both prompt templates and pipeline configurations were fixed in advance using separate development examples. All 170 commands were reserved exclusively for testing.

Model. All prompting strategies used OpenAI’s GPT-4.1-mini model for JOILang code generation. We kept the decoding settings fixed across all experiments so that differences between strategies arose only from prompting, service pre-mapping, and prompt-construction design rather than from stochastic decoding changes.

We adopted GPT-4.1-mini as the primary generation backbone because it provided the best practical balance of task quality, cost efficiency, and deployment feasibility for our multi-stage cloud pipeline. During pilot development, we explored alternative model options and decoding conditions, but these exploratory trials involved model-specific prompt adjustments and feasibility considerations, and thus were not treated as a controlled benchmark for cross-model or cross-decoding robustness. We therefore fixed GPT-4.1-mini and its decoding configuration for the main experiments to isolate the contribution of prompting and pipeline design under a realistic deployment-oriented setting. Unless otherwise noted, we use $w = 0.6$ as the default retrieval-weight setting in all downstream experiments reported in this section; this value was fixed during development, before the final test evaluation, and Section 5.6 reports the sensitivity of retrieval performance around this preselected operating point.

¹JOICommands-170 and the accompanying implementation artifacts are publicly available at <https://github.com/blueoxss/joillang-review-artifacts> (see also the Data availability statement).

Table 3

Schema summary of the JOICommands-170 dataset.

Field	Description
category	An integer label (1–9) indicating the logical complexity of the command; see Table 4 for definitions.
test_no	A unique integer identifier for each example, used to maintain consistent indexing and reproducibility. Experimental results are reported in the order of this index.
eng_command	The user-facing English command used as the input to the NL-to-JOILang generation pipeline.
connected_devices	A JSON-encoded list of devices available in the test scenario. It provides a snapshot of the environment's device registry, including types, tags, and basic attributes (e.g., cameras, lights, irrigation).
options	A JSON object encoding user-specific metadata such as address books. This field enables testing context-dependent commands (e.g., "Send email to Jangbo") requiring disambiguation.
gt1, gt2, gt3	Up to three ground truth JOILang scripts satisfying the input command. Each entry follows the format: {"name", "cron", "period", "code"}. These reference scripts were curated and cross-checked by two expert annotators, and multiple references are provided to capture valid variations in tagging and state logic.

Table 4

Detailed definitions of complexity categories in JOICommands-170. Categories are grouped by token length and control logic requirements.

Level	ID	Description	Key Characteristics
Level 1 Simple (< 10 toks)	1	One-shot Command	Single-device actuation or status query without complex logic. (e.g., "Turn on the TV", "Check temperature")
Level 2 Conditional (≤ 30 toks)	2	Simple Conditional	Multi-device control triggers with basic IF/ELSE conditions.
	3	Temporal State Flow	Logic dependent on state transitions or historical conditions (e.g., "if it opens" vs. "if open").
	4	Wait & Delay	Execution involving explicit blocking logic, implemented via <code>wait until</code> or <code>clock_delay</code> .
Level 3 Advanced (> 30 toks)	5	Complex Logic	Multiple conditions combined with Any/All quantifiers for group control.
	6	Loop & Break	Periodic execution (Cron/Period) requiring explicit termination (Break) conditions.
	7	Variables	Logic requiring variables declared with <code>:=</code> or updated with <code>=</code> for state tracking across iterations.
	8	User Tagging	Scoping via user-defined tags or composite device groups (e.g., "Living room lights").
	9	Context-Aware	Logic depending on external user context (e.g., user schedule, current time, weather).

Environment. All local pipeline stages (service pre-mapping, hybrid retrieval, prompt assembly, and deterministic validation) were executed on a single server equipped with two Intel Xeon Gold 6256 CPUs (3.60 GHz), 512 GB of RAM, and an NVIDIA RTX A6000 GPU (48 GB) used for the dense-retrieval embeddings, running Ubuntu 20.04 LTS. JOILang generation was performed through the OpenAI API.

Prompting Strategies. We compare the five strategies defined in Section 4.2: Baseline–Full Prompt, Baseline–CoT, Pre-map–Full Prompt, Pre-map–CoT+RAG (proposed), and Pre-map–CoD. For each strategy, we generated one candidate script per command over the full JOICommands-170 dataset and then applied the hybrid evaluation protocol defined in the next subsection to compute S_{DET} , pass rate, and token/latency statistics. This single-candidate protocol

Table 5

Comparison of prompting strategies. S_{DET} denotes the deterministic quality score, and Pass is the fraction of commands in JOICommands-170 that are rated as correct in our evaluation.

Strategy	Pass Rate	S_{DET}	Avg. Input Tokens	Avg. Latency (ms)
Baseline–Full Prompt	0.62	0.71	32,488	1,190
Baseline–CoT	0.78	0.87	44,837	1,331
Pre-map–Full Prompt	0.96	0.91	4,378	1,075
Pre-map–CoT+RAG (Proposed)	0.98	0.98	1,308	771
Pre-map–CoD	0.96	0.94	2,571	920

matches the deployment setting, in which the pipeline returns one script per command; we therefore report single-run results rather than averages over repeated samples.

5.2. Evaluation Metrics

All results were obtained using the end-to-end system described in Section 4 and were evaluated using a *hybrid evaluation protocol* that separates deterministic admissibility checking from benchmark-level semantic assessment.

Deterministic Quality S_{DET} . The deterministic (DET) module uses the JOILang parser, a type checker, and a Z3-based [47] constraint verifier to automatically detect syntax errors, type/unit/range violations, disallowed service calls, and non-terminating loops. If any fatal violation is detected, the candidate is assigned $S_{\text{DET}} = 0$. Otherwise, we compute an abstract syntax tree (AST)-based edit distance to the reference script set and normalize this value to obtain a score in $[0, 1]$.

Pass Rate. For benchmark-level semantic assessment, each candidate is compared against the expert-curated reference scripts described in Section 5.1 (fields `gt1`–`gt3` in Table 3). A candidate is first screened by the deterministic module. DET-passed candidates are then manually assessed against the expert reference set to determine whether the generated program is semantically consistent with the intended behavior. Ambiguous or borderline cases are resolved through expert adjudication. A command is counted as *Pass* only when the candidate is both deterministically admissible and semantically acceptable under this benchmark protocol.

Efficiency. To quantify operational cost, we report both the average number of input tokens and the end-to-end latency (in milliseconds), including prompt construction, LLM inference, and deterministic post-processing.

5.3. Comparison of Prompting Strategies

Table 5 summarizes the pass rate, deterministic quality S_{DET} , average number of input tokens, and end-to-end latency for all five strategies.

Effect of Service Pre-mapping. Baseline–Full Prompt and Baseline–CoT inject the entire service catalog into the context. As a result, they frequently select incorrect device/service combinations and produce ENUM or type errors, leading to the lowest S_{DET} and pass rates among all strategies (Table 5). In contrast, Pre-map–Full Prompt constrains the service set via Service Pre-mapping and improves S_{DET} from 0.71/0.87 to 0.91, and the pass rate from 0.62/0.78 to 0.96. This indicates that explicit search space restriction alone substantially increases the likelihood of generating code that is both executable and structurally close to the reference scripts.

Effect of Pre-map–CoT+RAG. Pre-map–CoT+RAG (Proposed) further refines the context by selectively injecting command-relevant grammar and example blocks and by guiding the model through a state-aware skeleton (Section 4.2.2). Among all strategies, it achieves (1) the highest pass rate (0.98), (2) the highest deterministic quality $S_{\text{DET}} = 0.98$, (3) the smallest average number of input tokens (1,308), and (4) the lowest end-to-end latency (771 ms) (Table 5). Compared to Pre-map–Full Prompt, both pass rate and S_{DET} improve, while tokens and latency decrease, yielding a strictly better accuracy–cost trade-off. Pre-map–CoD attains a lower pass rate (0.96) and S_{DET} (0.94), and incurs additional token and latency overhead relative to Pre-map–CoT+RAG because it verbalizes intermediate drafts.

Table 6

Pass rate comparison across difficulty levels for different prompting strategies.

Strategy	Level 1	Level 2	Level 3
Baseline–Full Prompt	0.95	0.72	0.48
Baseline–CoT	0.97	0.85	0.69
Pre-map–Full Prompt	1.00	0.98	0.94
Pre-map–CoT+RAG (Proposed)	1.00	0.98	0.97

Table 7

Representative-category component ablation. The table reports mean deterministic quality and mean semantic alignment for Category 1, Category 6, Category 7, and the macro average over the full dataset. Skel. denotes the state-aware skeleton.

Category	Mean Deterministic Quality				Mean Semantic Alignment			
	Full	w/o Skel.	w/o Time	w/o Pre-map	Full	w/o Skel.	w/o Time	w/o Pre-map
Cat. 1	0.990	0.974	0.967	0.932	0.846	0.821	0.801	0.756
Cat. 6	0.970	0.939	0.908	0.865	0.771	0.706	0.662	0.581
Cat. 7	0.960	0.921	0.889	0.838	0.724	0.668	0.613	0.537
Macro Avg.	0.980	0.951	0.923	0.883	0.782	0.733	0.691	0.615

Effect of Command Difficulty. Table 6 breaks down the pass rate by difficulty level for four representative strategies. All four handle Level 1 commands well (0.95–1.00), but the gap widens sharply as complexity increases: Baseline–Full Prompt drops to 0.48 on Level 3, whereas Pre-map–CoT+RAG retains 0.97. This trend is consistent with the design goal of suppressing service-grounding and temporal-logic errors through service pre-mapping and state-aware skeleton generation.

Taken together, the results in Tables 5 and 6 indicate that the proposed Pre-map–CoT+RAG strategy offers the most favorable balance between reliability and efficiency in our benchmark setting.

5.4. Component-wise Analysis of the Proposed Pipeline

To isolate the contribution of individual pipeline components without inflating the experimental budget, we evaluated representative categories from JOICommands-170: Category 1 (simple control), Category 6 (loop-and-break), and Category 7 (variable/state-tracking), together with a macro average over all categories. We compare the full system against three targeted ablations: *w/o State-Aware Skeleton Generation*, *w/o Time-Related Prompt*, and *w/o Pre-mapping*. This design exposes both the best-case and hardest-case regimes while avoiding category cherry-picking via the macro-average row. Here, *semantic alignment* denotes a continuous score in $[0, 1]$ that measures how closely the condition–action structure of a generated program matches the expert reference scripts; unlike the binary pass decision, it penalizes partial mismatches and is therefore systematically lower than the pass rate.

Table 7 shows that the full system consistently outperformed all three ablated variants across representative categories. In Category 1, the differences were modest: the full system achieved mean deterministic quality and semantic alignment of 0.990 and 0.846, respectively, whereas removing the state-aware skeleton, time-oriented prompt blocks, or pre-mapping reduced performance to 0.974/0.821, 0.967/0.801, and 0.932/0.756. The gap widened on the harder categories. In Category 6, removing pre-mapping caused the largest degradation, reducing semantic alignment from 0.771 to 0.581, while removing the state-aware skeleton reduced deterministic quality from 0.970 to 0.939. A similar pattern was observed in Category 7, where the full system achieved 0.960 deterministic quality and 0.724 semantic alignment, compared with 0.838/0.537 without pre-mapping. The macro-average row confirms the same trend over the full dataset: the full system obtained 0.980 deterministic quality and 0.782 semantic alignment, consistent with the S_{DET} of Table 5 and outperforming all ablations. These results indicate that pre-mapping is the most critical component for accurate service grounding, while state-aware skeleton planning and time-oriented rule injection are especially important for temporally complex commands.

Table 8

Comparison of the proposed auto-extracted VALUE/FUNCTION-split representation with two alternatives: a *Python subset* that uses a curated subset of Python service definitions for context, and an *Auto list (no split)*. Positive deltas for Pass and S_{DET} indicate improvement; negative deltas for tokens and latency indicate reduction (better).

Context Form	ΔPass	ΔS_{DET}	$\Delta\text{Tokens (\%)}$	$\Delta\text{Latency (\%)}$
Python subset	-0.10	-0.07	+89%	+22%
Auto list (no split)	-0.04	-0.05	+33%	+14%

Table 9

Sensitivity analysis for the retrieval weight w in Eq. (1).

w	Recall@5	Pass	S_{DET}
0.4	0.974	0.967	0.972
0.6	1.000	0.980	0.980
0.8	0.993	0.975	0.977

5.5. Representation Ablation: Service-Context Forms

Building on the strategy-level comparison in Table 5, we next examine how the representation of the injected device/service context affects performance while fixing the prompting setup to the best-performing configuration, Pre-map-CoT+RAG. In this ablation, all experiments share the same retrieval configuration and translation pipeline; only the form of the injected device/service context is varied.

To focus on representation effects rather than absolute score levels, Table 8 reports relative deltas with respect to the automatically extracted VALUE/FUNCTION-split context, which serves as the reference representation in our pipeline. We compare two alternative context forms against this reference: (i) *Python subset*, where a compact subset of Python service definitions is manually curated to include only command-relevant services, and (ii) *Auto list (no split)*, where the system automatically extracts a structured device/service list but does not separate the VALUE and FUNCTION roles.

The relative trends in Table 8 indicate that role-aware structuring of the injected service context provides a consistent reliability gain beyond retrieval-space restriction alone. Compared with a compact Python-based subset and an auto-extracted list without role separation, the VALUE/FUNCTION-split representation yields the most stable accuracy–efficiency trade-off. For example, using the Python subset context lowers the pass rate by 0.10 and S_{DET} by 0.07 while nearly doubling the token count relative to the split list. The no-split variant also underperforms the split context, with a 0.04 pass-rate drop and a 0.05 S_{DET} drop, along with higher token usage and latency. This representation-level effect helps explain why, under an otherwise fixed Pre-map-CoT+RAG pipeline, explicitly encoding which services are intended for conditions (VALUE) versus actions (FUNCTION) leads to the strongest overall performance, as observed in Table 5.

5.6. Sensitivity Analysis for the Retrieval Weight w

Table 9 shows that the retrieval weight w is stable over a broad operating range. Recall@5 changed from 0.974 at $w = 0.4$ to 1.000 at $w = 0.6$ and 0.993 at $w = 0.8$, while the downstream pass rate remained within a narrow band (0.967, 0.980, and 0.975). Deterministic quality showed the same pattern, varying only from 0.972 to 0.980 and 0.977. This suggests that the hybrid retriever does not depend on brittle tuning of w , and that the selected operating point provides a robust balance between lexical and embedding-based evidence. The default $w = 0.6$ was fixed during development, before the final test evaluation; in this sweep it also attains the best values on all three measures, supporting the preselected operating point used in all other experiments (Section 5.1). Note that we do not sweep the prompt-block depth k_{prompt} separately, since it is set adaptively from command length and complexity (Section 4.2.1), and the pre-mapping shortlist is capped at $N_{\text{service}} \leq 12$ services (Section 4.1), which bounds the injected context in all configurations. Recall@5 in Table 9 is measured over the top five entries of the device ranking, whereas Recall@10 in Table 2 is measured over the full $k_{\text{device}} = 10$ ranking.

5.7. Evaluation Scope and Deployment-Oriented Configuration

The present experiments are intentionally centered on a single deployment-oriented generation backbone with fixed decoding settings. This design allows differences between strategies to be attributed to prompt construction and pipeline structure rather than to model substitution or sampling variation, but it does not by itself establish robustness across model families or decoding setups.

The reverse-translation and feedback-memory mechanisms described in Section 4.3 were active in the deployed pipeline but were not separately quantified in this benchmark; their evaluation as a user-facing workflow is left to future deployment studies. The next section discusses these findings and the remaining limitations.

6. Discussion and Limitations

The experiments support the proposed NL-to-DSL pipeline within the evaluated smart-home and smart-office setting. The main findings are as follows:

- **Service Pre-mapping** decomposes a natural language command into VALUE (condition) and FUNCTION (action) roles and selects only the relevant devices and services. This constrains the search space and sharply reduces decoder confusion. The component ablation (Table 7) confirms that removing pre-mapping causes the largest performance drop, especially on harder categories.
- **Structure-aware prompt construction matters beyond retrieval alone.** The proposed RAG-Prompt strategy adapts grammar explanations and examples to each command, while the state-aware skeleton makes temporal and transition logic explicit before syntax instantiation. Together, these choices improve both pass rate and deterministic quality while sharply reducing token usage.
- **The strongest results come from combining schema narrowing, structured reasoning, and deterministic validation.** Pre-map-CoT+RAG delivers the best balance between accuracy, latency, and prompt cost among the evaluated strategies.
- **Reverse translation and feedback memory** expose intent mismatches, support iterative clarification, and enable reuse of accepted interaction patterns over time. Rather than being presented as a completed usability study, this mechanism is positioned as a user-side inspection and correction workflow for improving alignment between user intent and generated JOILang scripts.

In practical terms, these results suggest a simple deployment guideline: Pre-map-CoT+RAG should be preferred whenever the atomic prompt-block infrastructure is available; when only the service catalog is maintained, Pre-map-Full Prompt already recovers most of the reliability gain; and full-catalog prompting should be avoided in token- or latency-constrained IoT deployments.

The scope of the present study is intentionally limited. The evaluation focuses on user-centric smart homes and smart offices and therefore does not establish readiness for mission-critical industrial or safety-critical IoT. Likewise, the results do not constitute a proof of cross-DSL or cross-runtime generalization. JOILang-specific elements (such as the grammar, service schema, tag-binding semantics, and runtime policies) would need to be redefined for another target environment, even if some higher-level architectural ideas transfer.

More broadly, the results suggest a reliability-oriented NL-to-DSL pattern in which schema-aware retrieval narrows the action space, structured prompt assembly injects task-relevant rules and examples, and deterministic validation constrains the final program before deployment. The empirical evidence for this pattern remains tied to JOILang and JOICommands-170.

Several limitations also remain. First, benchmark-level semantic assessment currently relies on expert-curated references together with manual inspection of ambiguous cases, which is reliable but not yet scalable to much larger datasets or fully automated evaluation. Second, the current system targets soft real-time temporal patterns; richer industrial policies and stronger timing guarantees remain outside the scope of this study. Third, JOILang generation is still cloud-hosted, whereas execution and deterministic validation occur at the edge. Supporting stronger privacy and offline operation will require further progress in compact on-device models and deployment tooling. Finally, broader validation across alternative commercial APIs, compact local/open models, and varied decoding settings, together with more scalable human-in-the-loop evaluation protocols, remains an important direction for future work.

7. Conclusion

This paper studies reliable natural-language-to-IoT program generation using JOILang as a service-level executable representation. The proposed pipeline combines service pre-mapping, retrieval-augmented prompt construction, state-aware skeleton generation, deterministic validation and repair, and reverse translation for user-side inspection. On JOICommands-170, the Pre-map–CoT+RAG configuration improves pass rate from 0.62 to 0.98 and deterministic quality from 0.71 to 0.98 relative to a full-prompt baseline, while reducing average input tokens by approximately 96% and end-to-end latency by approximately 35%. Component and representation analyses further show that service pre-mapping, time-related rule injection, skeleton construction, and VALUE/FUNCTION splitting contribute to the observed gains.

These results support the use of service-aware context narrowing and validation-guided generation for user-centric smart environments. The conclusion is limited to the evaluated JOILang setting, cloud-hosted LLM configuration, and smart-home/smart-office benchmark. Future work should extend the evaluation to compact local models, additional IoT runtimes, stronger privacy-preserving deployments, and larger-scale semantic assessment protocols.

Data availability

The JOICommands-170 dataset, prompt templates, evaluation scripts, and implementation artifacts used in this study are publicly available at <https://github.com/blueoxss/joilang-review-artifacts>. A demonstration video of the end-to-end pipeline is provided as supplementary material.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used a generative AI assistant (Anthropic Claude) to assist with language editing, LaTeX formatting, and the organization of experiment scripts. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] A. Varol, N. Hossein Motlagh, M. Leino, S. Tarkoma, J. Virkki, Creation of AI-driven smart spaces for enhanced indoor environments – a survey, *Internet of Things* 36 (2026) 101876.
- [2] B. Ur, M. Pak Yong Ho, S. Brawner, J. Lee, S. Mennicken, N. Picard, D. Schulze, M. L. Littman, Trigger-action programming in the wild: An analysis of 200,000 IFTTT recipes, in: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ACM, 2016, pp. 3227–3231. doi:10.1145/2858036.2858556.
- [3] T.-H. K. Huang, A. Azaria, J. P. Bigham, InstructableCrowd: Creating IF-THEN rules via conversations with the crowd, in: *Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems*, ACM, 2016, pp. 1555–1562. doi:10.1145/2851581.2892502, cHI EA '16.
- [4] E. Attoh, B. Signer, From proprietary to high-level trigger-action programming rules: A natural language processing approach, *arXiv preprint arXiv:2310.15024*, 2023. URL: <https://arxiv.org/abs/2310.15024>, preprint; accessed 21 July 2026.
- [5] M. Lubbers, P. Koopman, R. Plasmeijer, Writing Internet of Things applications with task-oriented programming, *arXiv preprint arXiv:2212.04193*, 2022. URL: <https://arxiv.org/abs/2212.04193>, preprint; accessed 21 July 2026.
- [6] G. Aguzzi, N. Farabegoli, M. Viroli, A language-based approach to macroprogramming for IoT systems through large language models, *ACM Transactions on Internet of Things* 6 (2025) 1–30.
- [7] W3C Web of Things Working Group, Web of Things (WoT) Thing Description 1.1, W3C Recommendation, World Wide Web Consortium, 2023. URL: <https://www.w3.org/TR/wot-thing-description11/>, published 5 December 2023; accessed 21 July 2026.
- [8] Connectivity Standards Alliance, Matter Specification, Version 1.4, Standard specification, Connectivity Standards Alliance, 2024. URL: <https://csa-iot.org/developer-resource/specifications-download-request/>, accessed 21 July 2026.
- [9] F. Corno, L. De Russis, A. Monge Roffarello, A high-level semantic approach to end-user development in the Internet of Things, *International Journal of Human-Computer Studies* 125 (2019) 41–54.
- [10] SmartThings, Capabilities — smartthings developer documentation, Developer documentation, 2025. URL: <https://developer.smartthings.com/docs/devices/capabilities/>, accessed 21 July 2026.
- [11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, *arXiv preprint arXiv:2107.03374*, 2021. URL: <https://arxiv.org/abs/2107.03374>, preprint; accessed 21 July 2026.
- [12] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Auteume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Goyal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, O. Vinyals, Competition-level code generation with AlphaCode, *Science* 378 (2022) 1092–1097.
- [13] E. Kim, T. Son, S. Ha, A novel hierarchical edge-based architecture for service oriented IoT, *Internet of Things* 24 (2023) 100939.

- [14] MySmaX, JOI vs. traditional IoT platforms, Technical white paper, 2023. URL: <https://mysmax.gitbook.io/mysmax/joi-vs.-traditional-iot-platforms>, accessed 21 July 2026.
- [15] Home Assistant, Home assistant documentation, Developer documentation, 2026. URL: <https://www.home-assistant.io/docs/>, accessed 21 July 2026.
- [16] OpenJS Foundation, Node-RED documentation, Developer documentation, 2026. URL: <https://nodered.org/docs/>, accessed 21 July 2026.
- [17] F. Corno, L. De Russis, A. Monge Roffarello, HeyTAP: Bridging the gaps between users' needs and technology in IF-THEN rules via conversation, in: Proceedings of the International Conference on Advanced Visual Interfaces, ACM, 2020, pp. 30:1–30:9. doi:10.1145/3399715.3399905, aVI 2020.
- [18] Y. Gao, K. Xiao, F. Li, W. Xu, J. Huang, W. Dong, ChatIoT: Zero-code generation of trigger-action based IoT programs, Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 8 (2024) 103:1–103:29.
- [19] S. Zhang, J. Zhai, L. Bu, M. Chen, L. Wang, X. Li, Automated generation of LTL specifications for smart home IoT using natural language, in: Design, Automation and Test in Europe Conference and Exhibition, IEEE, 2020, pp. 622–625. doi:10.23919/DATE48585.2020.9116374, DATE 2020.
- [20] T. Scholak, N. Schucher, D. Bahdanau, PICARD: Parsing incrementally for constrained auto-regressive decoding from language models, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2021, pp. 9895–9901. doi:10.18653/v1/2021.emnlp-main.779.
- [21] M. Ahn, A. Brohan, N. Brown, et al., Do as I can, not as I say: Grounding language in robotic affordances, in: Proceedings of the 6th Conference on Robot Learning, volume 205 of *Proceedings of Machine Learning Research*, PMLR, 2023, pp. 287–318. URL: <https://proceedings.mlr.press/v205/ichter23a.html>, coRL 2022.
- [22] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, A. Zeng, Code as policies: Language model programs for embodied control, in: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, London, United Kingdom, 2023, pp. 9493–9500. doi:10.1109/ICRA48891.2023.10160591.
- [23] L. Beurer-Kellner, M. Fischer, M. T. Vechev, Prompting is programming: A query language for large language models, Proceedings of the ACM on Programming Languages 7 (2023) 1946–1969.
- [24] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, et al., Code llama: Open foundation models for code, arXiv preprint arXiv:2308.12950, 2023. URL: <https://arxiv.org/abs/2308.12950>, preprint; accessed 21 July 2026.
- [25] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, et al., Purple llama CyberSecEval: A secure coding benchmark for language models, arXiv preprint arXiv:2312.04724, 2023. URL: <https://arxiv.org/abs/2312.04724>, preprint; accessed 21 July 2026.
- [26] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, H. Nori, JSONSchemaBench: Evaluating constrained decoding with LLMs on efficiency, coverage and quality, in: ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models, Vancouver, Canada, 2025. URL: <https://openreview.net/forum?id=FK0aJqKoio>, iCML 2025 Workshop.
- [27] F. Raspanti, T. Ozcelebi, M. Holenderski, Grammar-constrained decoding makes large language models better logical parsers, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, Volume 6: Industry Track, Association for Computational Linguistics, 2025, pp. 485–499. URL: <https://aclanthology.org/2025.acl-industry.34/>. doi:10.18653/v1/2025.acl-industry.34.
- [28] R. Wishart, Grammar constraints on LLMs, Blog post, 2023. URL: <https://ryder.dev/blog/grammar-constraints-on-llms/>, accessed 21 July 2026.
- [29] K. Park, J. Wang, T. Berg-Kirkpatrick, N. Polikarpova, L. D'Antoni, Grammar-aligned decoding, in: Advances in Neural Information Processing Systems, 2024. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html, neurIPS 2024.
- [30] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, 2020, pp. 9459–9474.
- [31] F. Li, P. Fang, Z. Shi, A. Khan, F. Wang, D. Feng, W. Wang, X. Zhang, Y. Cui, CoT-RAG: Integrating chain of thought and retrieval-augmented generation to enhance reasoning in large language models, arXiv preprint arXiv:2504.13534, 2025. URL: <https://arxiv.org/abs/2504.13534>, preprint; accessed 21 July 2026.
- [32] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-Refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 46534–46594. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.
- [33] J. Cho, D. Kang, H. Kim, G. G. Lee, Self-correcting code generation using small language models, in: Findings of the Association for Computational Linguistics: EMNLP 2025, Association for Computational Linguistics, Suzhou, China, 2025, pp. 2345–2368. URL: <https://aclanthology.org/2025.findings-emnlp.127/>. doi:10.18653/v1/2025.findings-emnlp.127.
- [34] O. Debauche, S. Mahmoudi, A. Guttadauria, A new edge computing architecture for IoT and multimedia data management, Information 13 (2022) 89.
- [35] Amazon Web Services, AWS IoT Greengrass V2 documentation, Developer documentation, 2024. URL: <https://docs.aws.amazon.com/greengrass/v2/developerguide/>, accessed 21 July 2026.
- [36] Microsoft, Azure IoT edge documentation, Developer documentation, 2025. URL: <https://learn.microsoft.com/en-us/azure/iot-edge/>, accessed 21 July 2026.
- [37] E. Al-Masri, I. N. Subramanian, SOLAR: Illuminating LLM performance in API discovery and service ranking for edge AI and IoT, Internet of Things 32 (2025) 101630.

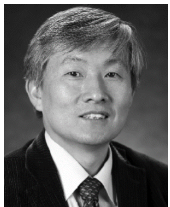
- [38] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, F. Wei, Text embeddings by weakly-supervised contrastive pre-training, arXiv preprint arXiv:2212.03533, 2022. URL: <https://arxiv.org/abs/2212.03533>, preprint; accessed 21 July 2026.
- [39] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (2009) 333–389.
- [40] Mixedbread AI, mxbai-embed-xsmall-v1 model card, Hugging Face model card, 2024. URL: <https://huggingface.co/mixedbread-ai/mxbai-embed-xsmall-v1>, accessed 21 July 2026.
- [41] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei, Multilingual E5 text embeddings: A technical report, arXiv preprint arXiv:2402.05672, 2024. URL: <https://arxiv.org/abs/2402.05672>, preprint; accessed 21 July 2026.
- [42] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, arXiv preprint arXiv:2402.03216, 2024. URL: <https://arxiv.org/abs/2402.03216>, preprint; accessed 21 July 2026.
- [43] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, volume 35, 2022, pp. 24824–24837.
- [44] S. Xu, W. Xie, L. Zhao, P. He, Chain of draft: Thinking faster by writing less, arXiv preprint arXiv:2502.18600, 2025. URL: <https://arxiv.org/abs/2502.18600>, preprint; accessed 21 July 2026.
- [45] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, Y. Wang, Skeleton-of-thought: Prompting LLMs for efficient parallel generation, in: International Conference on Learning Representations, 2024. URL: <https://openreview.net/forum?id=mqVgBbNCm9>, ICLR 2024.
- [46] C. Song, S. Kang, Snip-Cache: A code snippet caching system for LLM-based command-driven IoT systems, Internet of Things 36 (2026) 101852.
- [47] L. de Moura, N. Bjørner, Z3: An efficient SMT solver, in: Tools and Algorithms for the Construction and Analysis of Systems, Springer, 2008, pp. 337–340.



Mingi Jeong received the M.S. degree in computer science and engineering from Seoul National University, Seoul, Republic of Korea. He is currently pursuing the Ph.D. degree in the Department of Computer Science and Engineering at Seoul National University. He is also a researcher at the Electronics and Telecommunications Research Institute (ETRI), Daejeon, Republic of Korea. His research interests include natural-language-driven IoT automation, domain-specific languages, retrieval-augmented generation, and large-language-model-based code generation.



EunJin Jeong received the Ph.D. degree in computer science and engineering from Seoul National University, Seoul, Republic of Korea. He is currently a postdoctoral researcher with Seoul National University. His research interests include automatic robot software synthesis and embedded machine learning.



Soonhoi Ha (Fellow, IEEE) received the B.S. and M.S. degrees in electronics engineering from Seoul National University, Seoul, Republic of Korea, in 1985 and 1987, respectively, and the Ph.D. degree in electrical engineering and computer science from the University of California at Berkeley, Berkeley, CA, USA, in 1992. He is currently a professor with Seoul National University. His research interests include hardware/software codesign of embedded systems, embedded machine learning, and the Internet of Things. He served as the General Chair of ESWEEK in 2018 and is a member of ACM.